西北工业大学实验报告

课程名称:高级语言程序设计实验

实验课次周次:第四次（第十二周）

实验地点:实验大楼 107

学生姓名：张洛宾

学　　号:2024302696

实验时间:2026-05-23

提交时间:2025-05-29

成绩评定：（满分为 100 分） 指导教师评语： 指导教师签名：

目　录

1　UML 实验七 包图 2

1.1　实验内容（要求、算法、步骤和方法） 2

1.2　UML 结果文件说明 2

1.3　UML 实验结果 2

2　实验八 活动图 4

2.1　实验内容（要求、算法、步骤和方法） 4

2.2　UML 结果文件说明 4

2.3　UML 实验结果 4

3　UML 实验九 组件图 8

3.1　实验内容（要求、算法、步骤和方法） 8

3.2　UML 结果文件说明 8

3.3　UML 实验结果 8

4　UML 实验十 UML 综合实验 10

4.1　实验内容（要求、算法、步骤和方法） 10

4.2　UML 结果文件说明 10

4.3　UML 实验结果 10

5　实验总结 13

5.1　实验中遇到的问题以及解决方法 13

5.2　实验体会 13

1 UML 实验七 包图

1.1 实验内容（要求、算法、步骤和方法）

请参见实验指导书。

1.2 UML 结果文件说明

本次实验没有 UML 文件

1.3 UML 实验结果

实验内容与步骤:

1. 包是 UML 中的分组事物（Grouping Things），用于将模型中的相关元素（如类、用例、组件等）逻辑分组，形成更高层次的模块化结构。
2. UML 中有三种方式表示包,标准文件夹图标,构造型表示法,嵌套表示法
3. 依赖关系,泛化关系。
4. UML 中包与包之间的依赖关系具有单向性、弱耦合性和传递性的特点。这种关系表示一个包中的元素临时性地使用另一个包中的元素，属于一种松散的、非结构化的关联。依赖关系具有动态灵活性，允许被依赖的包在不影响依赖方的情况下进行替换，但需要注意避免循环依赖。
5. 避免出现循环依赖的关键是合理规划包结构和依赖方向,可通过依赖倒置原则将共享功能抽象为接口包（如订单处理和支付包共同依赖交易接口包），采用层级化分包确保单向依赖（如表现层→业务逻辑层→数据访问层），引入中间包存放公共功能（如用户管理和权限包通过公共工具包交互），必要时重构功能边界合并紧密耦合的包（如将相互依赖的库存包和订单包合并为交易核心包），或采用事件驱动架构通过消息总线实现间接通信（如日志包和监控包通过消息总线交互）。
6. 建立 UML 包图时需要注意以下几个关键问题：首先，要确保包的功能具有高内聚性，每个包应该只包含功能紧密相关的元素，避免将不相关的元素强行组合在一起；其次，要严格控制包之间的依赖关系，保持单向依赖并避免循环依赖，可以通过引入接口包或中间包来解耦；第三，要注意包的

粒度控制，既不能过大导致功能混杂，也不能过小增加管理复杂度；第四，合理使用包的可见性控制（public、private 等）来管理元素的访问权限；最后，在大型系统中要考虑包的层次结构设计，通过嵌套包来组织复杂的模块关系。同时，包图的命名应当清晰明确，反映其业务或技术职责，并保持与代码实现的一致性

7. B 依赖关系的检查应沿箭头方向逐级传递，不可逆向或跳跃。因此 B 是唯一正确描述依赖影响范围的选项。

8. A:

问题:包 B 与包 C 之间存在双向箭头（互相依赖），这会导致模块紧耦合，难以独立维护。包 A 直接依赖包 B，而包 B 又依赖包 E，此时包 A 对包 E 的依赖是间接传递的，但图中包 A 额外直接指向包 E，存在冗余。

解决:解除 B 与 C 的循环依赖：提取公共功能到新包（如 Common），让 B 和 C 共同依赖它。删除冗余箭头：移除包 A 直接指向包 E 的依赖，保留传递性依赖（A→B→E）。

B:

问题:包 B 同时依赖包 C 和包 D，而包 C 与包 D 又互相依赖，形成三角循环（B→C→D→B 的间接循环）。高层模块（如 A）应依赖稳定的底层模块（如 D），但图中底层包 D 反向依赖包 C，违反分层原则。

解决:将包 C 和包 D 的共享功能下沉到新包 Core，使 B 仅依赖 Core。确保依赖单向流动（A→B→Core，C/D 共同依赖 Core）。

2 实验八 活动图

2.1 实验内容（要求、算法、步骤和方法）

请参见实验指导书。

2.2 UML 结果文件说明

UML08.vpp---包含第八次实验的第六题的 UML 图

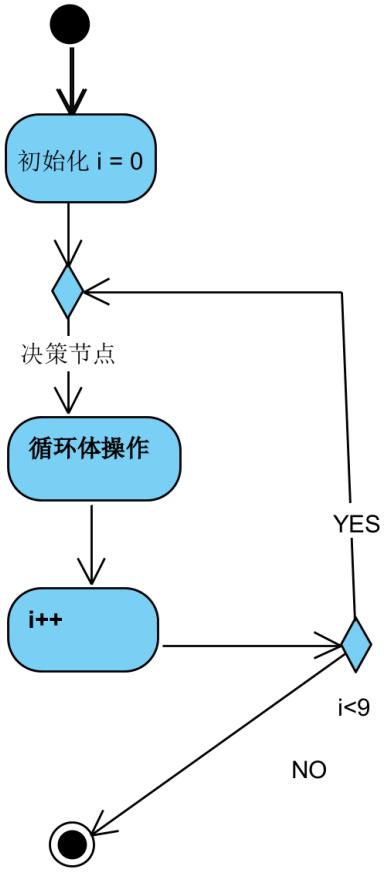
UML0801.vpp---包含第八次实验的分析与思考的 UML 图

2.3 UML 实验结果

1. 在 UML 中，活动（Activity）是指系统或对象执行的一个具体操作或行为步骤，它代表流程中的一个工作单元，可以是原子动作（如计算、发送消息）或复合活动（包含子活动）。活动在活动图（Activity Diagram）中通过圆角矩形表示，矩形内标注活动名称，例如"验证用户登录"。活动的执行可能涉及输入/输出（通过活动边上的引脚表示）、控制流（用箭头连接活动）或对象流（与数据交互）。
2. 活动图中包含以下元素及其表示方法：活动用圆角矩形表示，初始节点用实心圆表示，结束节点用同心圆表示，决策节点用菱形表示，分叉/汇合用粗横线表示，对象流用虚线箭头表示，控制流用实线箭头连接各元素，泳道用垂直或水平区域划分不同责任主体。
3. 分岔（Fork/Join）用粗横线表示，用于并发流程建模，表示同时执行多个活动（如多线程任务）；分支（Decision/Merge）用菱形表示，用于条件选择建模，表示根据条件执行不同路径（如 if-else 逻辑）。分岔适用于并行行为，分支适用于条件分支场景。
4. 活动图中监护条件用于控制流程分支（标注在决策箭头上，如[x>0]），决定是否执行某路径；改变时间（时间事件）则触发特定时间点的自动执行（如"after 2s"）。区别在于：监护条件基于数据判断流程走向，而改变时间基于时间触发动作，两者分别控制流程的空间选择性和时间触发机制。
5. 从"用户下订单"开始，用户需选择支付方式，若超时未支付则订单取消；支付成功后进入"收款"环节，系统生成送货单，供应商执行"送货"并循环

"修改订单项状态"，直至所有订单项送货完毕则标记"订单完成"。图中通过黄色活动节点、红色决策点和流程箭头完整展现了订单状态的正向流转和异常处理路径，特别是用虚线框标明了"修改订单项状态"这一需要循环执行的子流程

6.



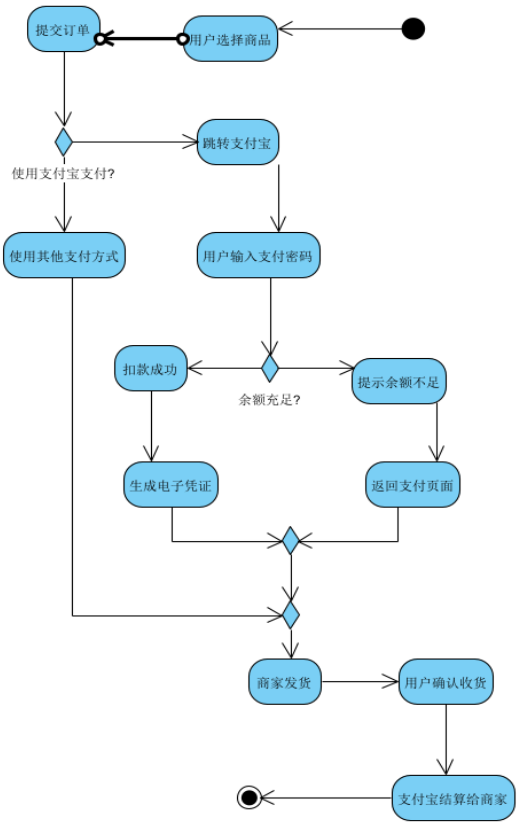
第六题的 UML 图，简要的实现了 for 循环

7. 从起始点开始执行"Activity1"后，流程分岔为两条并行路径（"Activity2"和"Activity3"），之后每条路径又各自分岔（"Activity2"→"Activity4"和"Activity3"→"Activity5"），最终在"Activity6"处汇合结束。图中红色分岔/汇合线明确标识了并发执行点，通过分析可知系统在运行时可能同时存在 4 个并发线程：当"Activity2"和"Activity3"并行执行时，它们各自的下游活动"Activity4"和"Activity5"也能继续并行处理，形成 2×2 的并发矩阵。所以设计的最大可能并发线程数是 4 个。

分析与讨论

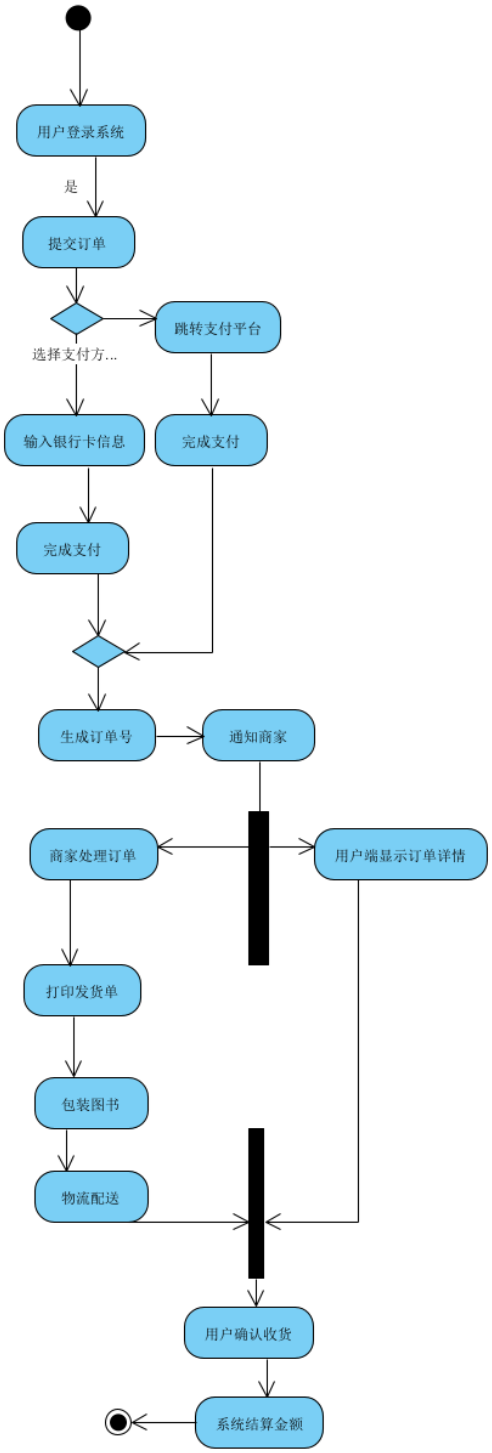
1. UML 在软件工程中作为标准化建模语言，通过可视化图形（如类图、时序图等）精确描述系统架构、行为和交互，其核心作用在于：1）统一开发

团队沟通语言，消除需求歧义；2）提前验证设计合理性，降低实现风险；3）生成代码框架提升开发效率。使用 UML 建模的必要性在于，它能将复杂业务逻辑转化为可迭代的模型，在需求分析、系统设计和文档维护阶段起到关键作用。



2.

支付宝网购流程从用户选择商品开始形成主线，在支付环节通过条件分支区分为支付宝支付（包含密码验证和余额检查）与其他支付方式两条路径，其中支付宝支付分支会处理余额不足的异常场景，整个流程贯穿完整的资金流动链条，从用户付款到商家发货，最终完成支付宝平台与商家的货款结算，形成闭环交易。



3.

采用循环结构处理图书选购过程

支付环节支持多支付平台选择

使用并行分支(fork)同步更新用户端和商家端状态

包含完整的物流配送和资金结算环节

3 UML 实验九 组件图

3.1 实验内容（要求、算法、步骤和方法）

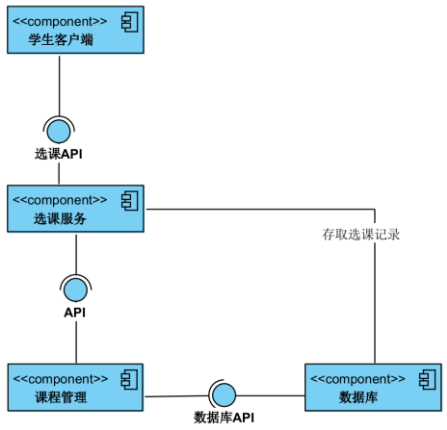
请参见实验指导书。

3.2 UML 结果文件说明

UML0901.vpp---包含第九次实验的实验目的中的 UML 图

3.3 UML 实验结果

实验目的中的组件图



1. 组件（Component）是可替换的系统模块单元，表示具有明确定义的接口的物理或逻辑代码模块。

作用包括：

1）封装内部实现细节，通过接口对外提供服务；

2）支持模块化开发，提升代码复用性；

3）描述系统物理部署结构（如.exe/.jar 文件的依赖关系）。

2. 在 UML 中，组件的核心要素包括名称（唯一标识）、接口（提供/需求两种类型）、端口（连接点）和内部实现结构。组件通过明确定义的接口（棒棒糖符号表示提供接口，半圆套接符号表示需求接口）与其他组件交互，其端口管理外部连接，而内部实现可包含类或子系统等具体逻辑，这些要素共同确保组件作为独立可替换模块的封装性和复用性。

3. 组件和类都是 UML 中的结构元素,类是逻辑设计单元,通过属性和方法描述对象,而组件是物理实现单元,通过接口封装可部署的模块。

主要区别在于类侧重抽象建模,组件侧重实现部署,类通过关联关系连接,组件通过接口实现契约交互。类可以成为组件的内部元素。

4.

- **矩形框带组件图标**（左侧带两个小矩形的矩形），表示标准可替换模块；
- **棒棒糖-套接符号**（圆形接口+半圆凹槽），分别代表组件提供的服务（棒棒糖）和需要的依赖（套接）；
- **端口符号**（小方块），用于精确控制组件间的连接点。这些表示法共同体现组件的核心特征——通过明确定义的接口封装功能,支持物理部署和模块化组装（如.dll/.jar 文件）。

5. 主要是两类接口,一个是提供接口,另外一个是需求接口。

提供接口用棒棒糖符号（圆圈连接实线）表示，标注组件对外公开的服务（如 PaymentService 接口），其他组件可直接调用该接口定义的操作。

需求接口用套接符号（半圆凹槽连接实线）表示，声明组件依赖的外部服务（如 Logger 接口），必须由其他组件实现该接口才能正常运作。

7 组件之间最常见的关系是接口依赖

服务提供关系,用棒棒糖符号表示

服务需求关系,用套接符号表示

分析与讨论

组件图主要适用于三类关键建模场景：

1）物理架构设计，清晰展示可执行文件（如.exe）、库文件（如.dll）等部署单元的依赖关系；

2）接口契约定义，通过棒棒糖/套接符号明确模块间的服务提供与需求关系（如微服务 API）；

3）系统集成规划，描述跨团队/跨系统的组件组装逻辑。

4 UML 实验十 UML 综合实验

4.1 实验内容（要求、算法、步骤和方法）

请参见实验指导书。

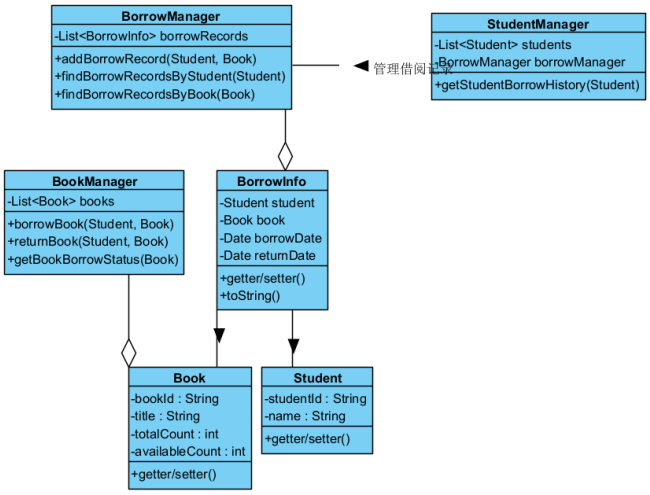
4.2 UML 结果文件说明

UML10.vpp---包含综合实验的所有 UML 图。

包括系统类图,借出流程的活动图,还书的顺序图,以及一本书的状态图

4.3 UML 实验结果

1. 系统类图



为了体现每本书的状态,应该对图书相关的类图做如下处理， 在 Book 类中添加状态属性(可借,已借出,丢失)， 在 BookManager 中增加状态校验逻辑（如仅 AVAILABLE 状态可借）， 借阅/归还时自动更新状态和库存数量（availableCount）

类图说明

BorrowInfo：关联学生和书籍，记录借还日期。

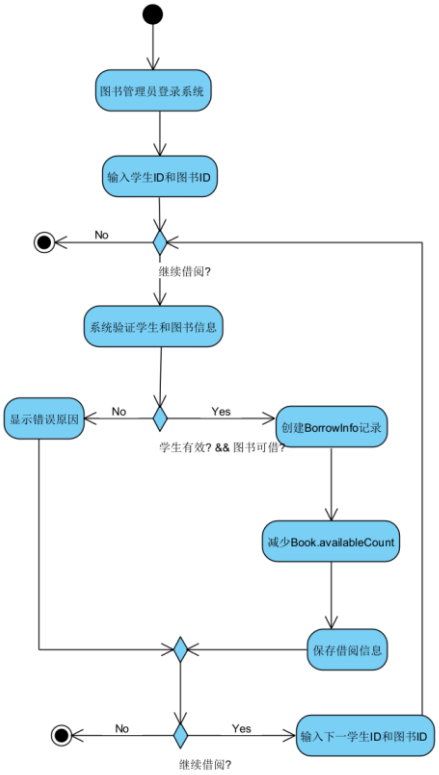
BorrowManager：集中管理所有借阅记录，支持按学生/书籍查询。

依赖关系：StudentManager 通过组合持有 BorrowManager 实例。

2. 借出工作流程的活动图

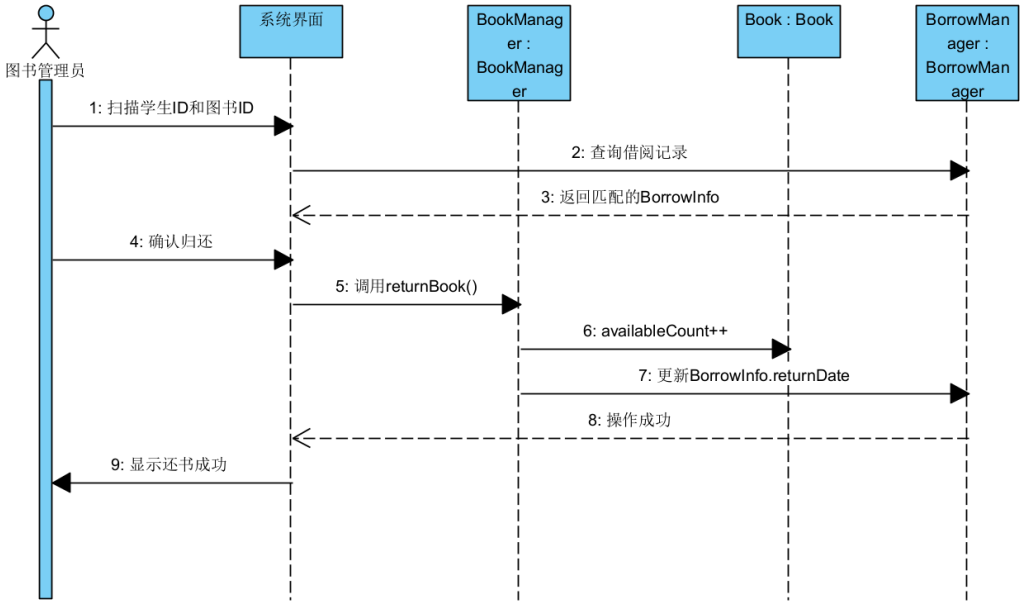
验证学生资格和图书库存。,库存不足或学生无效时终止流程。

支持连续借阅多本书。



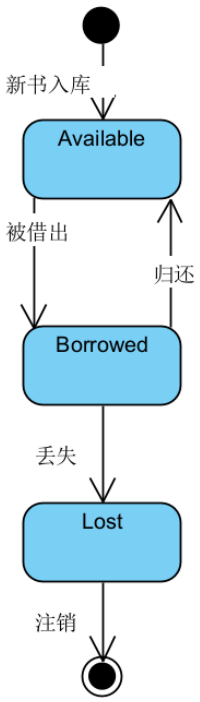
3. 还书的顺序图

管理员通过系统界面触发还书操作。BookManager 负责修改库存，BorrowManager 更新归还日期。



4. 一本图书的状态图

书有三个状态，可借，被借出，丢失。



5 实验总结

5.1 实验中遇到的问题以及解决方法

问题 1 在进行实验七包图的时候,判断包图的绘制是否合理出现问题,对于循环依赖和包中的元素的修改之后的依赖关系不够清楚。

解决 1 通过复习课上讲解的包图的循环依赖问题,以及组织包的各种原则之后,掌握了这部分知识。

问题 2 在进行实验八活动图的过程中,对于分支、监护条件、分岔、汇合和泳道等图形的画法还不够熟悉,对于各种关系的概念比较模糊

解决 2 通过复习课本知识和在 visual paradigm 中进行了简单的练习之后顺利掌握。

问题 3 进行实验 9 的过程中,我没有找到组件的棒棒糖图标和套接图标如何进行绘制,以及组件之间的关联关系和接口之间的关系。

解决 3 通过复习课本知识和查看 visual paradigm 的文档,发现在组件图中需要有 class 来提供 API。

问题 4 进行综合实验的过程中,对于整个系统的把握和各个 UML 图的关系的理解不够深刻,导致画图的过程中没有很好的兼顾各个图之间的关联。

解决 4 通过复盘整个图书管理系统的流程,理顺其中的关系,将不同关系通过不同的 UML 图呈现出来,并且保持其中的固有关联。

5.2 实验体会

通过这四次 UML 实验，对软件建模有了更深入的理解，刚开始接触 UML 时觉得这些图形符号很抽象，但随着实验的推进，我逐渐体会到 UML 在软件开发中的重要性。

在包图实验中，学会了如何用模块化思维组织代码结构。以前写 Java 项目时经常把所有类放在一个包里，现在明白了合理分包能显著提高代码的可维护性。特别是在设计"图书管理系统"时，通过将用户管理、借阅管理等功能划分到不同包中，代码结构变得清晰多了。

活动图实验让我掌握了业务流程的可视化方法。以前写程序时总是直接开始编码，现在会先用活动图梳理逻辑。比如在做 Java 实验时，我会先画出"用户登录验证"的活动图，这样能避免遗漏异常处理分支。分岔和汇合的概念也帮助我更好地理解了多线程编程。

组件图实验让我认识到接口设计的重要性。在实现 Java 的图书馆管理系统时，我特意将借阅功能抽象成独立组件，通过定义清晰的接口与其他模块交互。这种解耦设计使后期添加”续借功能时”只需修改少量代码。

综合实验,需要同时考虑类图、状态图、活动图等多个视图之间的关系。这让我明白软件开发不是简单的编码，而是需要全方位的设计思维。当我在 Java 实现中发现类图设计不合理时，通过修改 UML 模型就能提前发现问题，节省了大量调试时间。

这些实验不仅提升了建模能力，也培养了我的工程思维。在后续的程序设计当中,我也能通过学习到的 UML 建模的知识,先进行建模,再编写大型软件项目,这样能够合理化项目的布局与代码风格。为后续的更新迭代提供了便利。